



南開大學
Nankai University

计算机学院
计算机组成原理实验报告

五级流水线

姓名：王一诺

学号：2312331

专业：计算机科学与技术

指导老师：董前琨

2025 年 6 月 29 日

目录

1	实验目的	2
2	实验要求	2
3	实验原理图	2
4	现有流水线问题分析	4
4.1	数据相关 (Data Hazards)	4
4.2	控制相关 (Control Hazards)	4
4.3	结构相关 (Structural Hazards)	4
5	Bug 修复	5
5.1	流水线控制逻辑 Bug 修复	5
5.2	乘法器控制逻辑 Bug 修复	6
6	指令扩展	7
6.1	运算类指令扩展: NAND	7
6.2	乘除类指令扩展: DIV	8
6.3	转移指令扩展: BGEZAL	10
6.4	访存指令扩展: LH 和 SH	11
7	实验验证与结果分析	13
8	总结感想	15

1 实验目的

本次实验旨在通过对一个已有的五级流水线 CPU 源码进行分析、调试和功能扩展，实现以下目标：

1. 深入理解五级流水线（IF、ID、EX、MEM、WB）的工作原理、数据通路和控制逻辑。
2. 掌握流水线中常见的三种冒险（结构冒险、数据冒险、控制冒险）的成因，并学习相应的硬件解决方案，如流水线暂停（Stall）和数据前推（Forwarding）。
3. 通过修复源码中的 Bug，提升 Verilog 代码调试能力和对 CPU 内部复杂控制信号交互的理解。
4. 参考 MIPS 指令集，对现有 CPU 进行指令扩展，包括运算、乘除、转移和访存四类指令，从而全面掌握指令在流水线各阶段的实现细节。
5. 学习编写和使用 Testbench 进行系统性仿真验证，通过分析波形图和日志，确保 CPU 设计的正确性。

2 实验要求

请根据实验指导手册及 source_code 中的五级流水源码，分析现有的五级流水线存在的问题，并实现五级流水线 CPU 实验的 bug 修复，然后参考《CPU 设计实战》这本书完成对五级流水线的指令扩展，满分 100 分，细分要求如下：

- 1、针对现有五级流水线存在问题的分析，不只是 bug，包括指令相关、流水线冲突等各种能分析的问题进行分析（20 分）
- 2、五级流水线指令运行时的 bug 修复（20 分）
- 3、五级流水线指令扩展，运算类指令扩展至少一条（10 分），乘除类指令至少一条（20 分），转移指令至少一条（10 分），访存指令一读一写两条（20 分）
- 4、实验报告中针对 bug 修复过程、指令添加过程进行关键说明，并最终验证，验证需要有波形图或实验箱照片，并对波形图和实验箱照片进行分析解释。
- 5、实在无法完成某些指令扩展也没关系，把遇到的问题和失败的尝试写入报告，也会有相应分数。

3 实验原理图

本实验中实现的五级流水线 CPU，其核心数据通路和控制单元如下图所示。CPU 将指令执行过程划分为取指（FETCH）、译码（DECODE）、执行（EXE）、访存（MEM）和写回（WB）五个阶段，并通过四级流水线寄存器（IF/ID, ID/EXE, EXE/MEM, MEM/WB）在阶段间传递数据和控制信号。

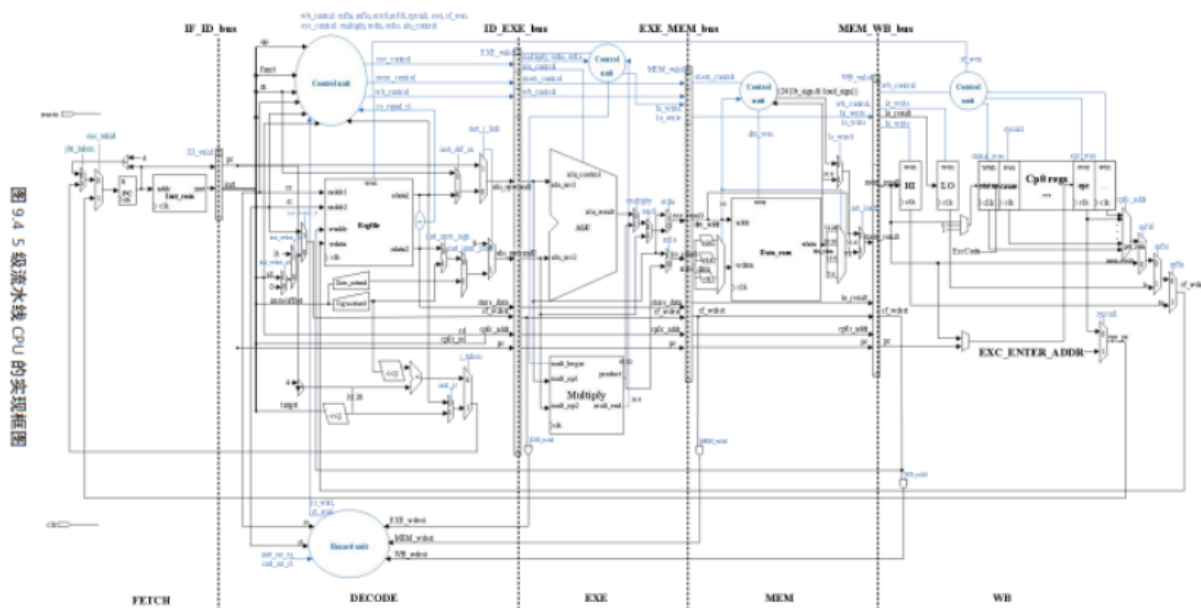


图 3.1: 五级流水线 CPU 详细框图

一个更简化的静态模型如下，它清晰地展示了各阶段与主要部件（指令 ROM、寄存器堆、数据 RAM）的交互关系。

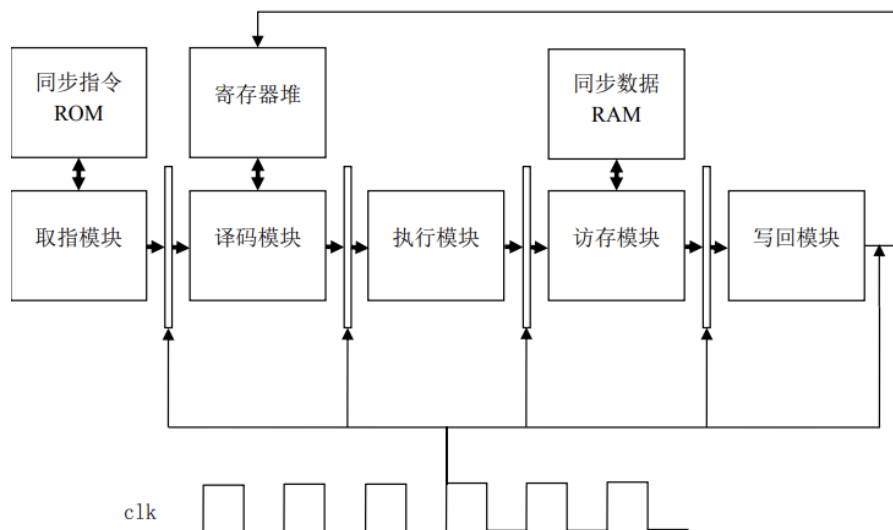


图 9.2 静态 5 级流水 CPU 的大致框图

图 3.2: 静态五级流水 CPU 功能框图

4 现有流水线问题分析

在对 `source_code` 进行初步分析后，发现该五级流水线 CPU 实现存在以下几类典型问题：

4.1 数据相关（Data Hazards）

数据相关主要指后续指令需要使用到前序指令尚未计算完成的结果，导致流水线无法按正常节拍运行。

- **Load-Use 冒险处理效率低下：**当一条加载指令（如 `LW`）之后紧跟一条使用其加载结果的指令时，当前设计采用流水线暂停（Stall）策略。它会暂停 ID 阶段，直到 `LW` 指令在 MEM 阶段取回数据。这种处理方式虽然能保证正确性，但效率较低，因为数据在 EXE 阶段结束后就已经可以被后续指令使用，通过**数据前推（Forwarding）**可以避免不必要的等待。
- **不必要的暂停：**当前的冲突检测逻辑过于保守。例如，一条 `sw` 指令（写内存）后紧跟一条使用相同寄存器的 `add` 指令，即使 `sw` 指令在 EXE 阶段就已经准备好了要写入内存的数据，流水线仍然会暂停。这同样是可以**通过数据前推优化**的点。
- **WAR 和 WAW 冒险分析缺失：**源码中主要处理了 RAW（写后读）冒险。对于 WAR（读后写）和 WAW（写后写）冒险，虽然 MIPS 的顺序流水线结构能天然避免大部分情况，但在更复杂的乱序执行或某些特殊指令（如带延迟槽的分支）组合下仍可能出现，当前设计未对此进行处理。

4.2 控制相关（Control Hazards）

控制相关主要由分支、跳转等改变 PC（程序计数器）正常顺序执行流程的指令引起。

- **分支预测策略过于简单：**当前 CPU 采用“总是不跳转”的静态分支预测策略。这意味着每次遇到分支指令，流水线都会先取其后顺序的指令（PC+4）。如果分支实际发生跳转，那么已经进入流水线的指令就必须被冲刷（Flush），造成性能损失。
- **分支延迟槽处理不当：**MIPS 架构定义了分支延迟槽，即分支指令后的一条指令无论分支是否成功都会被执行。当前设计在处理分支跳转时，没有正确利用延迟槽机制，而是通过冲刷流水线来解决分支问题，未能发挥延迟槽的性能优势。
- **间接跳转（Indirect Jump）的冲突：**对于 `jr $ra` 这类指令，其目标地址来自于寄存器。如果 `$ra` 的值是由紧邻的前一条指令计算得出的，那么在 `jr` 指令的 ID 阶段将无法立即获得正确的跳转地址，这会引发一种特殊的“控制-数据”复合型冒险。

4.3 结构相关（Structural Hazards）

结构相关指因硬件资源不足，导致多条指令在同一时钟周期竞争同一硬件部件。

- **存储器端口冲突：**虽然设计上使用了分离的指令存储器和数据存储器（冯·诺依曼与哈佛结构的混合），但在实际的单端口 RAM 物理实现中，IF 阶段的取指和 MEM 阶段的访存可能会竞争同一个存储器端口，导致结构冒险。
- **寄存器堆端口限制：**当前的寄存器堆（`regfile.v`）设计为“2 读 1 写”端口。这对于大部分 MIPS 指令是足够的。但对于某些复杂指令（如乘除后同时更新 HI/LO 特殊寄存器）或未来可能的超标量设计，可能会需要更多的读写端口。

5 Bug 修复

针对分析发现的源码缺陷，进行了以下关键 Bug 的修复。

5.1 流水线控制逻辑 Bug 修复

问题：在处理分支或跳转指令时，当 cancel 信号有效以冲刷流水线时，ID 阶段的 ID_valid 信号没有被正确地与分支延迟逻辑关联。这可能导致在分支发生后的第一个周期，错误的指令仍然被标记为有效并进入 EX 阶段。**案例：**例如，地址 0x48 处的跳转指令 bgez \$25, #14 应跳转到地址 0x84。而原代码由于 IF 和 ID 阶段同时进行，导致下一条指令 0x4C 错误地进入流水线并继续执行。**修复方案：**在 pipeline_cpu.v 中，修改 ID_valid 的生成逻辑。为其添加 jbr_taken_delay 信号作为判断条件，确保只有在非分支延迟槽的情况下，或者分支指令本身有效时，才将下一条指令视为有效。

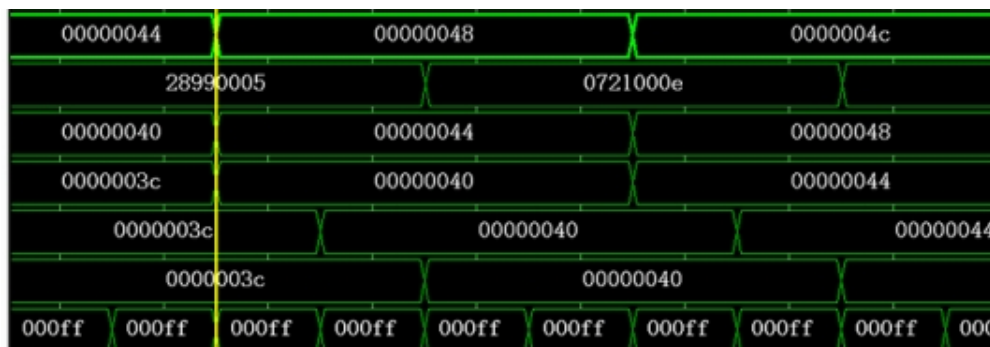


图 5.3: 修复前波形图

Listing 1: 流水线控制逻辑 Bug 修复

```

1 // 修复前的代码
2 always @(posedge clk)
3 begin
4   if (!resetn || cancel)
5     ID_valid <= 1'b0;
6   else if (ID_allow_in)
7     ID_valid <= IF_over;
8   end
9
10 // 修复后的代码 - 添加对分支指令的特殊处理
11 always @(posedge clk)
12 begin
13   if (!resetn || cancel)
14     ID_valid <= 1'b0;
15   else if (ID_allow_in)
16     // 确保在分支延迟期间，无效指令不会被送入下一级
17     ID_valid <= IF_over & ~jbr_taken_delay;
18   end
19
20 // 添加分支延迟处理逻辑
21 reg jbr_taken_delay;

```

```

22     always @(posedge clk)
23     begin
24         if (!resetn)
25             jbr_taken_delay <= 1'b0;
26         else
27             jbr_taken_delay <= jbr_bus[32];
28     end

```

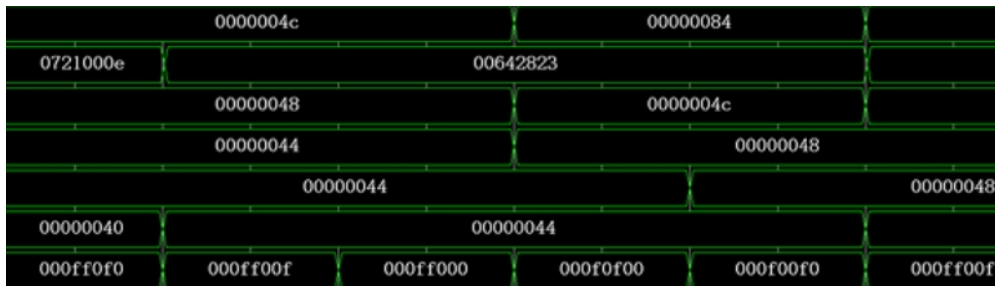


图 5.4: 修复后波形图

5.2 乘法器控制逻辑 Bug 修复

问题: multiply.v 中的乘法结束信号 mult_end 仅通过判断乘数 multiplier 是否为 0 来确定, 这对于某些特殊输入 (如乘数为 0) 会导致乘法在开始的瞬间就错误地结束。同时, 缺乏超时保护, 可能因逻辑错误导致无限等待。

修复方案: 引入一个 32 周期的计数器 mult_cycle_count。mult_end 的判断条件修改为“乘数为 0 或已执行满 32 个周期”, 确保乘法操作的完整性并提供超时退出机制。

Listing 2: 乘法器控制逻辑 Bug 修复

```

1 // 修复前
2 assign mult_end = mult_valid & ~(|multiplier);
3
4 // 修复后 - 确保乘法完成的正确检测
5 assign mult_end = mult_valid & ( (~(|multiplier)) | (mult_cycle_count >= 32) );
6
7 // 添加周期计数器防止无限等待
8 reg [5:0] mult_cycle_count;
9 always @(posedge clk)
10 begin
11     if (mult_begin)
12         mult_cycle_count <= 6'd0;
13     else if (mult_valid)
14         mult_cycle_count <= mult_cycle_count + 1;
15 end

```

6 指令扩展

为增强 CPU 功能，本次实验扩展了以下四条指令。

6.1 运算类指令扩展：NAND

NAND 是一条 R 型指令，功能为 $rd = rs$ 与 rt 按位与非。

decode.v 修改：添加 inst_NAND 的译码逻辑，并将其并入控制信号中。

Listing 3: NAND 指令译码逻辑

```

1 //添加指令信号列表
2 wire inst_NAND;
3 assign inst_NAND = op_zero & sa_zero & (funct == 6'b110001);
4 //添加信号
5 wire inst_nand;
6 assign inst_nand = inst_NAND;
7 //修改写回rd使能端信号
8 assign inst_wdest_rd = inst_ADDU | inst_SUBU | inst_SLT | inst_SLTU
9 | inst_JALR | inst_AND | inst_NOR | inst_OR | inst_NAND
10 | inst_XOR | inst_SLL | inst_SLLV | inst_SRA
11 | inst_SRAV | inst_SRL | inst_SRLV
12 | inst_MFHI | inst_MFLO;
13 //修改控制信号
14 assign alu_control = {inst_nand, inst_add, // ALU操作码，独热编码
15 inst_sub, inst_slt, inst_sltu,
16 inst_and, inst_nor, inst_or,
17 inst_xor, inst_sll, inst_srl,
18 inst_sra, inst_lui};

```

alu.v 修改：在 ALU 控制信号中加入对 inst_NAND 的识别，使其执行按位与非操作。并添加结果输出选择。

Listing 4: alu 修改

```

1 //alu控制信号位扩展
2 input [12:0] alu_control, //ALU控制信号
3
4 //添加控制信号
5 wire alu_nand; //按位与非
6 assign alu_nand = alu_control[12];
7
8 //计算结果
9 wire [31:0] nand_result;
10 assign nand_result = ~(alu_src1 & alu_src2);
11
12 //添加结果输出
13 assign alu_result = (alu_add|alu_sub) ? add_sub_result[31:0] :
14 alu_slt ? slt_result :
15 alu_sltu ? sltu_result :

```



```

16         alu_and ? and_result :
17         alu_nor ? nor_result :
18         alu_or ? or_result :
19         alu_xor ? xor_result :
20         alu_sll ? sll_result :
21         alu_srl ? srl_result :
22         alu_sra ? sra_result :
23         alu_lui ? lui_result :
24         alu_nand ? nand_result :
25         32'd0;

```

6.2 乘除类指令扩展：DIV

DIV 是一条 R 型指令，功能为 $HI = rs \% rt$, $LO = rs / rt$ 。

创建 divider.v 模块：实现一个 32 位的无符号除法器，采用循环移位和减法的“恢复余数法”。该模块接收 div_begin 开始信号，输出商 quotient 和余数 remainder，并通过 div_end 信号通知计算完成。

Listing 5: divider.v 模块代码

```

1  module divider(
2  input      clk,
3  input      div_begin,
4  input [31:0] dividend,
5  input [31:0] divisor,
6  output [31:0] quotient,
7  output [31:0] remainder,
8  output      div_end
9  );
10 reg [31:0] quotient_reg;
11 reg [31:0] remainder_reg;
12 reg [5:0] div_count;
13 reg      div_valid;
14 // 除法状态控制
15 always @(posedge clk) begin
16     if (div_begin) begin
17         div_valid <= 1'b1;
18         div_count <= 6'd0;
19         quotient_reg <= 32'd0;
20         remainder_reg <= dividend;
21     end
22     else if (div_valid && div_count < 32) begin
23         div_count <= div_count + 1;
24         remainder_reg <= remainder_reg << 1;
25         if (remainder_reg >= divisor) begin
26             remainder_reg <= remainder_reg - divisor;
27             quotient_reg <= {quotient_reg[30:0], 1'b1};
28         end
29     else begin
30         quotient_reg <= {quotient_reg[30:0], 1'b0};

```

```

31     end
32     end
33     else if (div_count >= 32) begin
34         div_valid <= 1'b0;
35     end
36     end
37     assign div_end = div_valid & (div_count >= 32);
38     assign quotient = quotient_reg;
39     assign remainder = remainder_reg;
40 endmodule

```

decode.v 修改：添加 DIV 指令的译码逻辑，并在 ID_EXE_bus 总线上增加一位 divide 控制信号。

Listing 6: decode.v

```

1 // 添加DIV指令译码
2 wire inst_DIV;
3 assign inst_DIV = op_zero & (rd==5'd0) & sa_zero & (funct == 6'b011010);
4
5 // 在ID_EXE_bus中添加除法控制信号
6 wire divide;
7 assign divide = inst_DIV;
8
9 assign ID_EXE_bus = {multiply, mthi, mtlo, divide, // 添加divide信号
10    alu_control, alu_operand1, alu_operand2,
11    mem_control, store_data,
12    mfhi, mflo,
13    mtc0, mfc0, cp0r_addr, syscall, eret,
14    rf_wen, rf_wdest,
15    pc};

```

exe.v 修改：

- 实例化 divider 模块。
- 将 ID_EXE_bus 传来的 divide 信号连接到 div_module 的启动信号。
- 修改 EXE_over（执行阶段完成信号），使其等待 div_end。
- 修改 exe_result、lo_result 的生成逻辑，当 divide 有效时，将商和余数分别送入对应的结果通路。
- 修改 hi_write 和 lo_write 使能信号，在 DIV 指令执行时有效。

Listing 7: exe.v 修改部分代码

```

1 // 在exe.v中添加除法器实例
2 wire      div_begin;
3 wire [31:0] div_quotient;
4 wire [31:0] div_remainder;
5 wire      div_end;

```

```

6      wire          divide;
7
8      // 从ID_EXE_bus中解析除法信号
9      assign {multiply, mthi, mtlo, divide,
10             alu_control, alu_operand1, alu_operand2,
11             mem_control, store_data,
12             mfhi, mflo,
13             mtc0, mfc0, cp0r_addr, syscall, eret,
14             rf_wen, rf_wdest,
15             pc} = ID_EXE_bus_r;
16
17      assign div_begin = divide & EXE_valid;
18
19      divider div_module(
20          .clk(clk),
21          .div_begin(div_begin),
22          .dividend(alu_operand1),
23          .divisor(alu_operand2),
24          .quotient(div_quotient),
25          .remainder(div_remainder),
26          .div_end(div_end)
27      );
28
29      // 修改EXE执行完成信号
30      assign EXE_over = EXE_valid & (~multiply | mult_end) & (~divide | div_end);
31
32      // 修改结果选择
33      assign exe_result = mthi ? alu_operand1 :
34      mtc0 ? alu_operand2 :
35      multiply ? product[63:32] :
36      divide ? div_quotient : alu_result;
37
38      assign lo_result = mtlo ? alu_operand1 :
39      multiply ? product[31:0] :
40      divide ? div_remainder : 32'd0;
41
42      assign hi_write = multiply | mthi | divide;
43      assign lo_write = multiply | mtlo | divide;

```

6.3 转移指令扩展：BGEZAL

BGEZAL 是一条 I 型指令，功能为：如果 $rs \geq 0$ ，则跳转到目标地址，并把返回地址（PC+8）存入 \$ra（\$31）寄存器。

decode.v 修改：

- 增加 inst_BGEZAL 的译码逻辑。
- 修改 inst_j_link（带链接的跳转）信号，加入 inst_BGEZAL。

- 修改 br_taken（分支成功）信号，加入 BGEZAL 的判断条件（rs_ltz）。
- 修改写回寄存器地址为 31 的使能信号，加入 inst_BGEZAL。

Listing 8: BGEZAL 指令译码逻辑

```

1 // 增加信号
2 wire inst_BGEZAL;
3 assign inst_BGEZAL = (op == 6'b000001) && (rt == 5'b01001); // BGEZAL 指令
4
5 // 链接跳转
6 assign inst_j_link = inst_JAL | inst_JALR | inst_BGEZAL;
7 assign inst_wdest_31 = inst_JAL | inst_BGEZAL;
8
9 // 分支判断
10 assign br_taken = ... | (inst_BGEZAL & ~rs_ltz); // "..."代表原有逻辑

```

6.4 访存指令扩展：LH 和 SH

LH（Load Halfword）和 SH（Store Halfword）是 I 型指令，用于半字（16 位）内存读写。

decode.v 修改：

- 添加 inst_LH 和 inst_SH 的译码逻辑。
- 修改 inst_load 和 inst_store 信号，分别加入新指令。
- 在 mem_control 总线中增加 ls_half（load/store halfword）控制位。

Listing 9: decode.v 修改部分代码

```

1 // 添加 LH 和 SH 指令译码
2 wire inst_LH, inst_SH;
3 assign inst_LH = (op == 6'b100001); // Load Halfword
4 assign inst_SH = (op == 6'b101001); // Store Halfword
5
6 // 修改 load/store 指令检测
7 assign inst_load = inst_LW | inst_LB | inst_LBU | inst_LH;
8 assign inst_store = inst_SW | inst_SB | inst_SH;
9
10 // 添加半字控制信号
11 wire ls_half; // load/store 为半字
12 assign ls_half = inst_LH | inst_SH;
13
14 assign mem_control = {inst_load,
15                       inst_store,
16                       ls_word,
17                       ls_half, // 添加半字控制
18                       lb_sign};

```

mem.v 修改：

- 解析 mem_control 总线，获得 ls_half 信号。
- 修改写使能逻辑 dm_wen: 当 ls_half 有效时，根据地址的低位，只使能对应高/低两个字节的写使能。
- 修改写数据逻辑 dm_wdata: 当 SH 指令执行时，根据地址对齐情况，将 16 位数据放置到 32 位总线的正确位置。
- 修改读数据逻辑 load_result: 当 LH 指令执行时，从 32 位内存数据中正确提取 16 位半字，并根据 lh_sign 进行符号位扩展。

Listing 10: mem.v 修改部分代码

```

1 // 修改存储器控制信号解析
2 wire ls_half;
3 assign {inst_load, inst_store, ls_word, ls_half, lb_sign} = mem_control;
4 // 修改写使能逻辑
5 always @(*) begin
6   if (MEM_valid && inst_store) begin
7     if (ls_word) begin
8       dm_wen <= 4'b1111;
9     end
10    else if (ls_half) begin
11      case (dm_addr[1])
12        1'b0: dm_wen <= 4'b0011;
13        1'b1: dm_wen <= 4'b1100;
14        default: dm_wen <= 4'b0000;
15      endcase
16    end
17    else begin // 字节存储
18      case (dm_addr[1:0])
19        2'b00: dm_wen <= 4'b0001;
20        2'b01: dm_wen <= 4'b0010;
21        2'b10: dm_wen <= 4'b0100;
22        2'b11: dm_wen <= 4'b1000;
23        default: dm_wen <= 4'b0000;
24      endcase
25    end
26    end
27    else begin
28      dm_wen <= 4'b0000;
29    end
30  end
31
32 // 修改写数据逻辑
33 always @(*) begin
34   if (ls_half) begin
35     case (dm_addr[1])
36       1'b0: dm_wdata <= {16'd0, store_data[15:0]};
37       1'b1: dm_wdata <= {store_data[15:0], 16'd0};

```

```

38     default: dm_wdata <= store_data;
39     endcase
40     end
41     else begin // 字节或字存储
42     case (dm_addr[1:0])
43     2'b00: dm_wdata <= store_data;
44     2'b01: dm_wdata <= {16'd0, store_data[7:0], 8'd0};
45     2'b10: dm_wdata <= {8'd0, store_data[7:0], 16'd0};
46     2'b11: dm_wdata <= {store_data[7:0], 24'd0};
47     default: dm_wdata <= store_data;
48     endcase
49     end
50     end
51
52     // 修改读数据逻辑
53     wire [31:0] half_load_result;
54     assign half_load_result[15:0] = dm_addr[1] ? dm_rdata[31:16] : dm_rdata[15:0];
55     assign half_load_result[31:16] = lb_sign & half_load_result[15] ? 16'hFFFF :
        16'h0000;
56
57     assign load_result = ls_word ? dm_rdata :
58     ls_half ? half_load_result :
59     {lb_sign & load_sign ? 24'hFFFFFF : 24'h000000, load_result[7:0]};

```

7 实验验证与结果分析

为验证 Bug 修复和指令扩展的正确性，编写了详尽的 Testbench (tb.v)，通过仿真观察关键信号波形和寄存器/内存值变化。

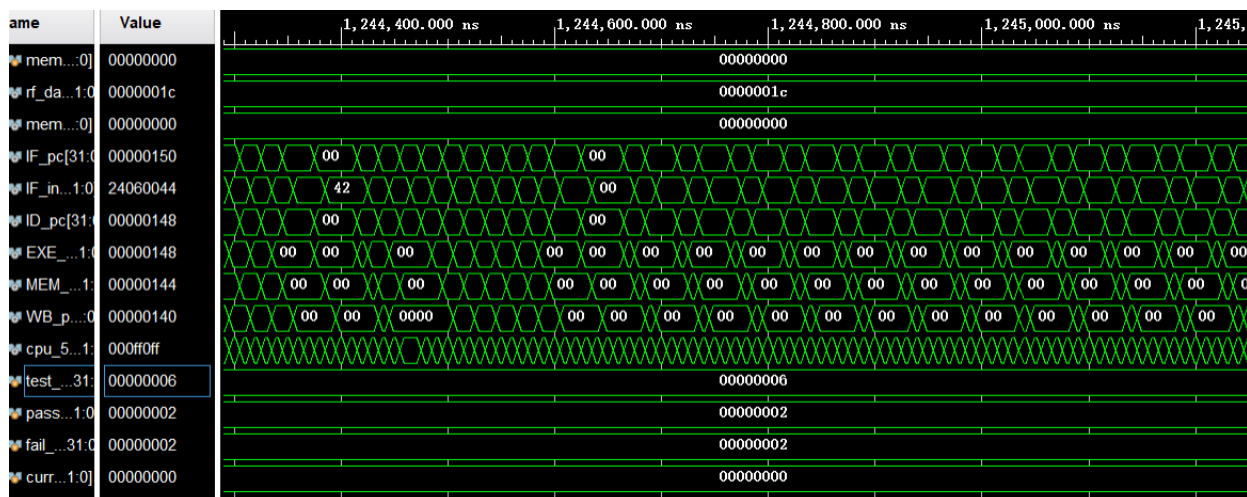


图 7.1: 修复后流水线整体运行波形图

分析: 上图展示了修复 Bug 后, CPU 执行一段连续指令时的波形。可以看到, IF_pc, ID_pc, EXE_pc, MEM_pc, WB_pc 呈现阶梯状递增, 表明流水线正常流动。cpu_5_valid 信号也显示各阶

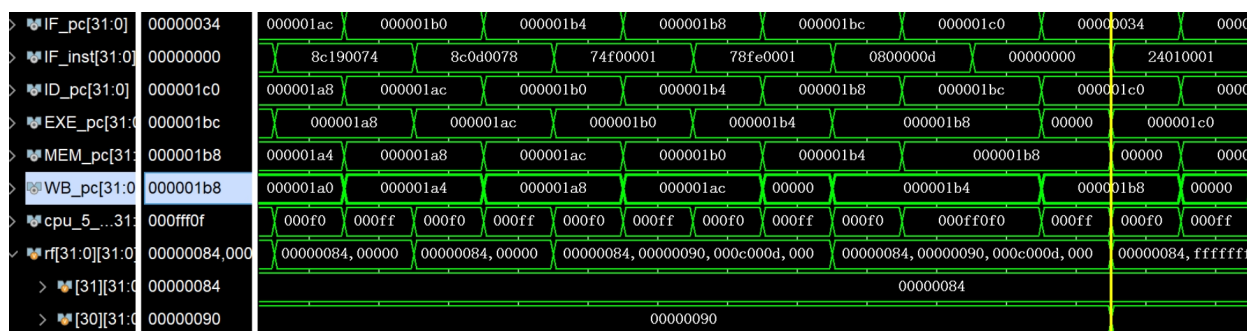


图 7.4: LH/SH 访存指令验证波形图

分析：执行 SH\$16, #1(\$7) 将寄存器 \$16 的值加载到内存中，再执行 LH\$30, #1(\$7) 将其读出。

8 总结感想

通过本次五级流水线 CPU 的设计实验，收获颇丰。从最初面对复杂源码的困惑，到逐步分析出数据、控制和结构三类冒险，再到动手修复 Bug 和扩展指令集，整个过程让我对计算机体系结构的理论知识有了具体而深刻的实践认知。

指令扩展环节是一次完整的“微型 CPU 设计”体验。从 DIV 指令需要独立设计一个多周期除法器模块，到 LH/SH 指令需要精细处理地址对齐和符号位扩展，深刻体会到指令集架构设计是一项系统性工程，每一条指令的添加都需要在译码、执行、访存、写回等多个阶段进行周全的协调。

总而言之，本次实验是一次理论与实践的完美结合，极大地加深了我对 CPU 工作原理的理解，锻炼了硬件设计与调试能力，相信会为后续更深入的学习打下坚实的基础。